54b. Redis Explained with Diagrams: What, Where It Runs, How It's Used

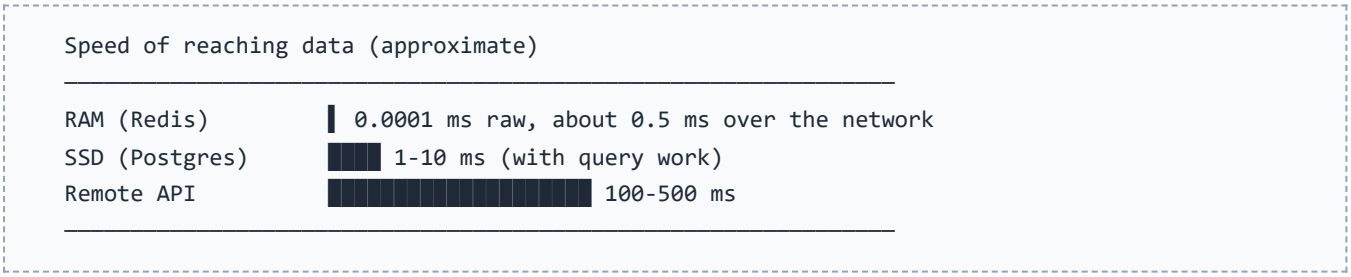
A diagram-first companion to **#49 (Redis basics + NestJS cache)** and **#54a (stores and TTL)**. Also uses **#57 (BullMQ)** and **#63 (WSL 2)**.

> **In one line:** Redis is a **super-fast database that keeps data in RAM** as **key → value** pairs. Apps use it for **caching, sessions, OTPs, rate limits, queues, leaderboards, locks and live messages**. It answers in **under 1 millisecond**.

PART 1: What Redis is

1. Real-life example: the desk drawer vs the store room

> **Postgres / Mongo** = the **store room** in the basement. It holds everything safely, but walking there takes time (disk, about 5-50 ms).
> **Redis** = the **drawer of your desk**. Small, but what you need often is **right at your hand** (RAM, about 0.1-1 ms).
> You keep the **original** in the store room, and a **copy of frequently used things** in the drawer.



2. Name and history

- **REDIS = REmote DIctionary Server**: a "dictionary" (key → value) you reach over the network.
- Made in **2009** by **Salvatore Sanfilippo** (Italy) to speed up his real-time web analytics startup.
- Became one of the most-used databases in the world (Twitter/X, GitHub, Stack Overflow, Instagram).
- 2024: the licence changed, and the community forked it as **Valkey** (used by AWS ElastiCache). Same commands. 2025: Redis 8 became open source again (AGPL).

3. Key → value: the basic shape

```
KEY (a name, a string)          VALUE (the data)                          TTL
─────────────────────           ──────────────────────────────────        ──────

"otp:9876543210"          ->    "4821"                                    300 s
"student:1"               ->    { name: "Ravi", marks: 80 }  (hash)       none
"session:abc123"          ->    { userId: 7, role: "admin" }              1 day
"leaderboard"             ->    Ravi 980, Asha 870, Kiran 760 (zset)      none
"bull:emails:wait"        ->    [job1, job2, job3]           (list)       none
```

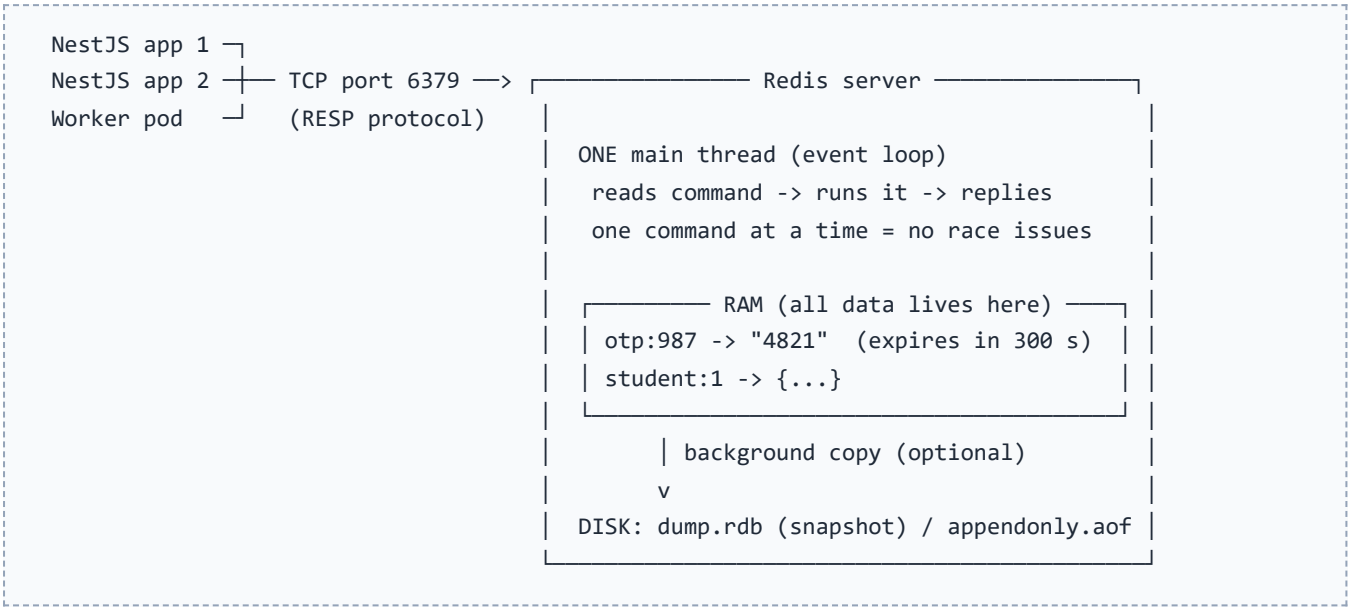
Key naming tip: use `type:id`, like `student:1` or `otp:98765`. The colon groups keys like folders.

4. Data types (Redis is more than a cache)

Type	Looks like	Commands	Used for
String	`"4821"`, JSON text, a number	`SET GET INCR EXPIRE`	Cache, OTP, counters
Hash	Small object: field → value	`HSET HGET HGETALL`	User profile, session
List	Ordered list, push/pop at the ends	`LPUSH RPOP LRANGE`	Queues, recent items
Set	Unique items, no order	`SADD SISMEMBER`	Online users, tags, "already seen"
Sorted Set	Unique items + a score, auto-sorted	`ZADD ZRANGE ZREVRANGE`	Leaderboards, rankings, scheduling
Stream	Append-only event log	`XADD XREAD`	Event logs, chat history
Pub/Sub	Channels (not stored)	`PUBLISH SUBSCRIBE`	Live notifications

PART 2: How Redis works inside

5. The architecture



6. Why it's so fast

Reason	Meaning
In RAM	No disk reads for normal commands
Single-threaded core	No locks or thread fights. Each command is tiny (microseconds), so one thread handles 100,000+ commands per second.
Event loop + non-blocking I/O	Same idea as Node.js (#56): thousands of connections on one thread
Simple data structures	Most commands are O(1) (instant, whatever the size)
Atomic commands	`INCR`, `SET NX` can't be interrupted, which makes them perfect for counters and locks

Single thread means one slow command blocks everyone, like CPU work in Node. Never run `KEYS *` on a big production Redis. Use `SCAN`.

7. If it's in RAM, what happens on a restart? (Persistence)

```
RDB (snapshot)                          AOF (append-only file)
──────────────                          ──────────────────────
every N minutes:                        every write command is logged:
photo of ALL data -> dump.rdb           SET a 1 / INCR b / DEL c ... -> appendonly.aof

restart: load the photo                 restart: replay the log
may lose the last few minutes           loses at most about 1 second
small file, fast restart                bigger file, safer

Common in prod: BOTH (RDB for backups + AOF for safety)
Pure cache: often NONE (the data can be rebuilt from Postgres)
```

8. When RAM is full (Eviction)

```
maxmemory 1gb reached ──> new SET arrives ──> which key to throw out?
                                                  |
     allkeys-lru    -> least recently used key   (best for caches)
     volatile-ttl   -> key closest to expiry
     noeviction     -> refuse new writes (error)  (needed for queues like BullMQ)
```

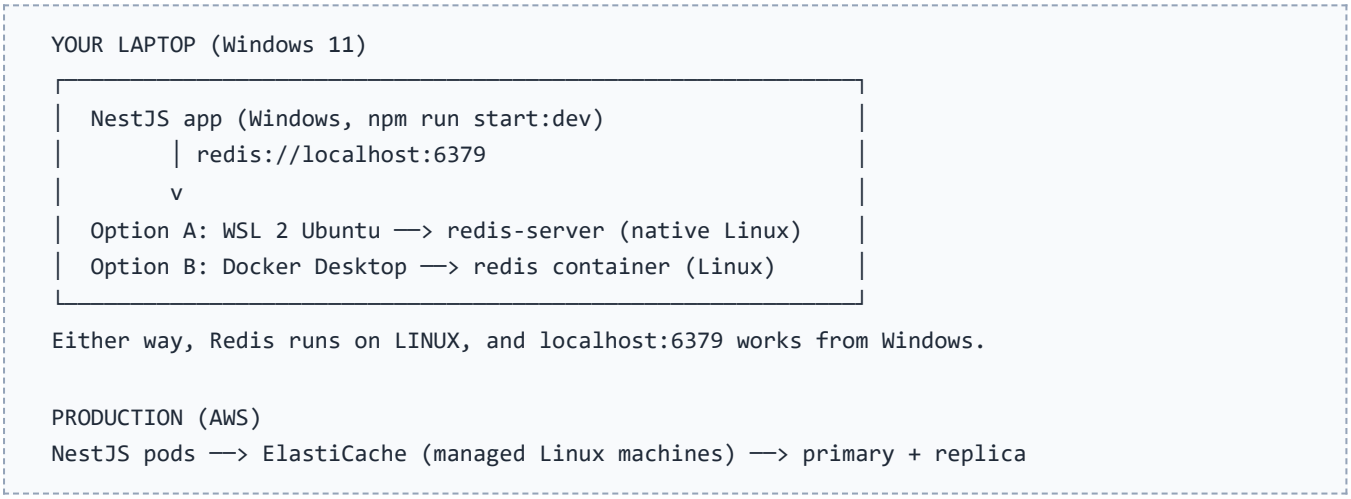
9. TTL: auto-delete (recap of #54a)

```
SET otp:98765 4821 EX 300
   |
   t=0s     key exists        TTL otp:98765 -> 300
   t=200s   key exists        TTL otp:98765 -> 100
   t=300s   key DELETED       GET otp:98765 -> (nil)
Redis deletes it by itself, so no cron job is needed (#58)
```

PART 3: Which OS Redis runs on

10. Redis is a Linux program

OS	Support	How to run
Linux (Ubuntu, Debian, Alpine)	**Official, best**. Production runs here.	`sudo apt install redis-server`
macOS	Works (for development)	`brew install redis`
Windows	**No official native build**	Use **WSL 2** (#63), **Docker**, or Memurai (a Windows port)
Cloud	Managed, so you don't touch the OS	AWS ElastiCache (Valkey/Redis), Upstash, Redis Cloud, Azure Cache



Option A: install in WSL 2 Ubuntu (your new setup)

```
wsl -d Ubuntu
sudo apt update && sudo apt install -y redis-server
sudo service redis-server start
redis-cli ping                 # PONG = working
# from Windows, NestJS connects to redis://localhost:6379
```

Option B: Docker

```
docker run -d --name redis -p 6379:6379 redis:7
docker exec -it redis redis-cli ping     # PONG
```

11. Try it: redis-cli

```
redis-cli
127.0.0.1:6379> SET name "Gourish"
OK
127.0.0.1:6379> GET name
"Gourish"
127.0.0.1:6379> SET otp:98765 4821 EX 60     # expires in 60 s
127.0.0.1:6379> TTL otp:98765
(integer) 57
127.0.0.1:6379> INCR visits                  # counter: 1, 2, 3...
(integer) 1
127.0.0.1:6379> HSET student:1 name Ravi marks 80
127.0.0.1:6379> HGETALL student:1
127.0.0.1:6379> ZADD leaderboard 980 Ravi 870 Asha 760 Kiran
127.0.0.1:6379> ZREVRANGE leaderboard 0 2 WITHSCORES   # top 3
127.0.0.1:6379> DEL name
127.0.0.1:6379> SCAN 0 MATCH student:* COUNT 100        # safe "find keys"
```

11b. Basic commands cheat sheet (with output)

Open the shell with `redis-cli` (WSL) or `docker exec -it redis redis-cli` (Docker). Commands are **not case-sensitive** (`set` = `SET`), but **keys are** (`Name` and `name` are different keys).

Connect and check

```
redis-cli                          # connect to localhost:6379
redis-cli -h 10.0.0.5 -p 6379 -a mypassword    # connect to another server
PING                               -> PONG          (server is alive)
ECHO "hello"                       -> "hello"
SELECT 1                           -> OK            (switch to database 1; there are 0-15,
default 0)
EXIT                               (leave redis-cli)
```

Strings (text / numbers)

```
SET city "Pune"                    -> OK
GET city                           -> "Pune"
GET unknown                        -> (nil)         (key doesn't exist)
SET city "Goa" NX                  -> (nil)         (NX = only if NOT exists, so not changed)
SET country "India" XX             -> (nil)         (XX = only if it ALREADY exists)
MSET a 1 b 2 c 3                   -> OK            (set many at once)
MGET a b c                         -> 1) "1" 2) "2" 3) "3"
APPEND city " city"                -> (integer) 9   (new length)
STRLEN city                        -> (integer) 9
INCR views                         -> (integer) 1   (counter +1, creates it if missing)
INCRBY views 10                    -> (integer) 11
```

```
DECR views                          -> (integer) 10
INCRBYFLOAT price 2.5               -> "2.5"
GETDEL city                         -> "Pune city"   (read and delete in one step)
```

Keys and expiry (works for every type)

```
KEYS *                              -> all keys      (OK on your laptop, NEVER in production)
SCAN 0 MATCH user:* COUNT 100       -> safe step-by-step search (production-safe)
EXISTS city                         -> (integer) 1   (1 = yes, 0 = no)
TYPE views                          -> string
RENAME views pageviews              -> OK
DEL a b c                           -> (integer) 3   (how many were deleted)
UNLINK bigkey                       -> (integer) 1   (delete in the background, for big keys)

SET otp:98765 4821 EX 120           -> OK            (expires in 120 seconds)
SET token abc PX 5000               -> OK            (PX = milliseconds)
EXPIRE pageviews 60                 -> (integer) 1   (add a TTL to an existing key)
TTL otp:98765                       -> (integer) 115 (seconds left)
TTL pageviews                       -> ...
TTL city                            -> (integer) -1  (-1 = no expiry)
TTL deletedkey                      -> (integer) -2  (-2 = key doesn't exist)
PERSIST otp:98765                   -> (integer) 1   (remove the expiry, keep forever)
```

Hashes (an object with fields)

```
HSET user:1 name "Ravi" age 21 city "Pune"   -> (integer) 3   (fields added)
HGET user:1 name                    -> "Ravi"
HMGET user:1 name city              -> 1) "Ravi" 2) "Pune"
HGETALL user:1                      -> name Ravi age 21 city Pune
HINCRBY user:1 age 1                -> (integer) 22
HEXISTS user:1 email                -> (integer) 0
HKEYS user:1                        -> name, age, city
HLEN user:1                         -> (integer) 3
HDEL user:1 city                    -> (integer) 1
```

Lists (ordered, like a queue or stack)

```
RPUSH tasks "email" "pdf" "sms"   -> (integer) 3   (add at the RIGHT end)
LPUSH tasks "urgent"              -> (integer) 4   (add at the LEFT end)
LRANGE tasks 0 -1                 -> urgent, email, pdf, sms   (0 to -1 = all)
LLEN tasks                        -> (integer) 4
LPOP tasks                        -> "urgent"      (take from the left = queue FIFO)
RPOP tasks                        -> "sms"         (take from the right)
LINDEX tasks 0                    -> "email"
BLPOP tasks 30                    -> waits up to 30 s for an item (worker-style)
LTRIM recent 0 9                  -> keep only the latest 10 items
```

Sets (unique items, no order)

```
SADD online "ravi" "asha" "ravi"  -> (integer) 2   (duplicate "ravi" ignored)
SMEMBERS online                   -> ravi, asha
SISMEMBER online "asha"           -> (integer) 1
SCARD online                      -> (integer) 2   (count)
SREM online "asha"                -> (integer) 1
SADD java "ravi" "kiran"
SINTER online java                -> ravi          (in BOTH sets)
SUNION online java                -> ravi, kiran   (in EITHER set)
```

Sorted sets (unique items + score, auto-sorted)

```
ZADD marks 80 "ravi" 92 "asha" 67 "kiran"   -> (integer) 3
ZRANGE marks 0 -1 WITHSCORES      -> kiran 67, ravi 80, asha 92   (low to high)
ZREVRANGE marks 0 1 WITHSCORES    -> asha 92, ravi 80             (top 2)
ZINCRBY marks 15 "kiran"          -> "82"
ZSCORE marks "ravi"               -> "80"
ZREVRANK marks "asha"             -> (integer) 0   (rank 0 = first place)
ZRANGEBYSCORE marks 75 100        -> ravi, kiran, asha   (scores 75-100)
ZCARD marks                       -> (integer) 3
```

Pub/Sub (live messages: use 2 terminals)

```
# terminal 1
SUBSCRIBE news                    -> waiting for messages...
# terminal 2
PUBLISH news "Exam on Monday"     -> (integer) 1   (1 subscriber received it)
# terminal 1 now shows: "news" "Exam on Monday"
```

Transactions (all or nothing)

```
MULTI                             -> OK            (start)
DECRBY wallet:ravi 500            -> QUEUED
INCRBY wallet:asha 500            -> QUEUED
EXEC                              -> 1) ... 2) ... (both run together, nothing in between)
```

Server and debugging

```
INFO server                       -> version, OS, uptime
INFO memory                       -> used_memory_human: 1.02M
DBSIZE                            -> (integer) 12  (number of keys)
MONITOR                           -> live view of EVERY command (debugging only, Ctrl+C to
stop)
SLOWLOG GET 5                     -> the 5 slowest recent commands
CONFIG GET maxmemory              -> current RAM limit
CONFIG SET maxmemory-policy allkeys-lru     -> set the eviction policy
```

```
FLUSHDB                          -> delete ALL keys in the current DB  (dangerous!)
FLUSHALL                         -> delete ALL keys in ALL DBs         (very dangerous!)
```

> **5-minute practice:** `PING`, then `SET name you`, `GET name`, `SET otp 1234 EX 20` and watch `TTL otp` count down, then `INCR visits` three times, `HSET`/`HGETALL` a user, `ZADD` a mini leaderboard, and `SUBSCRIBE`/`PUBLISH` in two terminals.

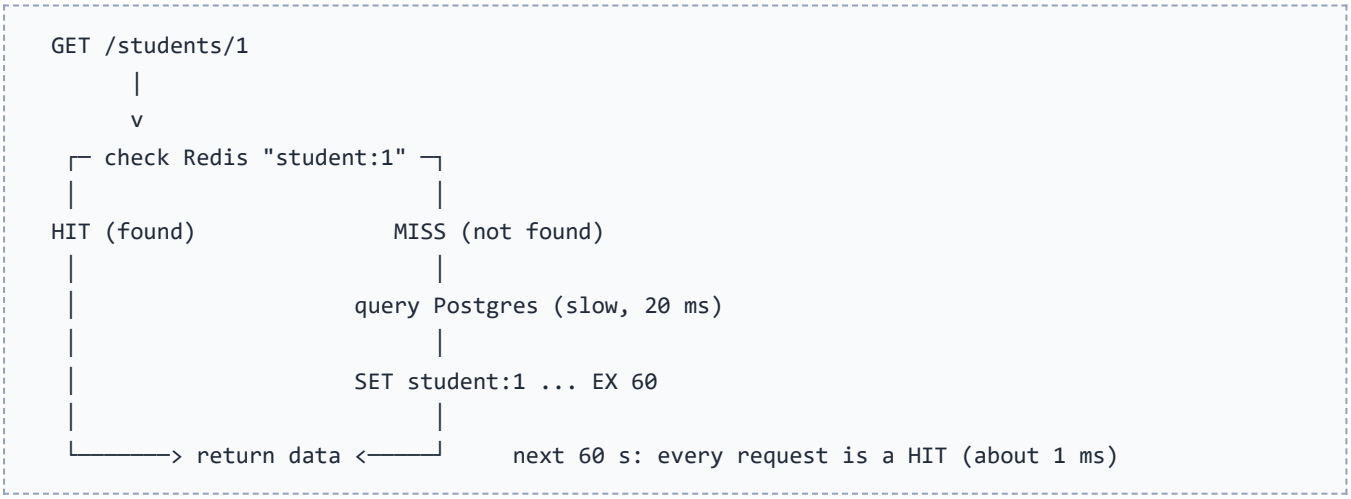
Same commands from NestJS (ioredis)

```
import Redis from 'ioredis';
const redis = new Redis('redis://localhost:6379');

await redis.set('city', 'Pune', 'EX', 60);          // SET city Pune EX 60
const city = await redis.get('city');               // GET city
await redis.incr('views');                          // INCR views
await redis.hset('user:1', { name: 'Ravi', age: 21 });   // HSET
const user = await redis.hgetall('user:1');         // HGETALL -> { name: 'Ravi', age: '21' }
await redis.zadd('marks', 80, 'ravi', 92, 'asha');  // ZADD
const top = await redis.zrevrange('marks', 0, 2, 'WITHSCORES');
await redis.del('city');                            // DEL
```

PART 4: How Redis is used (with diagrams)

12. Use 1: Cache (the most common)




```
async getStudent(id: number) {
  const cached = await this.redis.get(`student:${id}`);
  if (cached) return JSON.parse(cached);                    // HIT

  const student = await this.repo.findOneBy({ id });         // MISS -> DB
  await this.redis.set(`student:${id}`, JSON.stringify(student), 'EX', 60);
  return student;
```

```
}
// on update: await this.redis.del(`student:${id}`)  -> next read gets fresh data
```

What does `EX 60` do here? (simple words)

> **`EX 60` = "remember this copy for 60 seconds, then forget it."** After 60 s, Redis **deletes** the copy by itself. The next request then gets fresh data from Postgres and saves it again.

> **Sticky note example:** someone walks to the **store room** (Postgres) to find Ravi's phone number. That's slow. You write it on a **sticky note** (Redis) that **disappears by itself after 60 seconds**. Within 60 s, everyone reads the sticky note, which is fast. After 60 s the note is gone, so the next person walks to the store room again and writes a **new** note.

```
SET student:1 '{"name":"Ravi","marks":80}' EX 60
     |           |                          |
     key         value (the copy)           delete after 60 seconds
```

Second by second

```
 0 s   1st request -> Redis: nothing -> Postgres (20 ms) -> SET ... EX 60 -> reply
       timer starts (60 s)
 5 s   request     -> Redis: found -> reply (1 ms)     no Postgres
30 s   request     -> Redis: found -> reply (1 ms)     timer NOT reset by reading
59 s   request     -> Redis: found -> reply (1 ms)
60 s   Redis DELETES "student:1" by itself
       (nobody asks -> nothing happens, the key stays empty)
95 s   request     -> Redis: nothing -> Postgres (FRESH data) -> SET ... EX 60 again
       new 60 s timer starts
```

3 important points

Point	Meaning
1. The timer starts at SAVE	It starts the moment of `SET ... EX 60`, **not** on each request. Reading it 1,000 times doesn't add time.
2. Redis only DELETES, it doesn't UPDATE	After 60 s, Redis just removes the copy. It does **not** fetch new data from Postgres. The **next request** does that (it pays the slow 20 ms once and refills the cache for everyone).
3. The timer is strict	After 60 s, nobody can read it, even 1 ms late. `GET` returns `(nil)`. Redis checks the time on every read, so expired data is never returned.

What changes the timer

Action	Timer
`GET student:1` (reading, any number of times)	**No change**. It keeps counting down.
`SET student:1 ... EX 60` again	**Restarts** at 60
`SET student:1 ...` **without** EX	**Removed**: the key stays forever (careful!)
`EXPIRE student:1 120`	Changed to 120 s
`PERSIST student:1`	Removed: the key stays forever

Why 60 seconds (and not forever)?

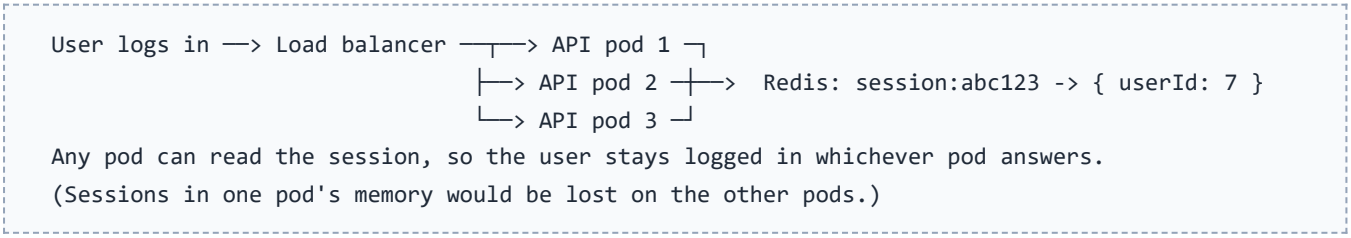
Without `EX 60`	**With `EX 60`**
The copy stays in Redis **forever**	The copy is deleted after **60 s**
If Ravi's marks change in Postgres, users see **old data forever**	Users see old data for **at most 60 s**, then fresh data
Redis memory keeps filling up	Old data cleans itself up

See it yourself in redis-cli

```
SET student:1 "Ravi" EX 60
TTL student:1      -> 60
GET student:1      -> "Ravi"
TTL student:1      -> 52        (8 s later: reading did NOT reset it)
...
TTL student:1      -> -2        (after 60 s: gone)
GET student:1      -> (nil)
```

One line: saved, then kept for 60 s, then **deleted**. The **next request** brings fresh data and saves it again for another 60 s.

13. Use 2: Sessions shared by many servers



14. Use 3: OTP with auto-expiry

```
POST /otp/send    -> SET otp:98765 4821 EX 300   -> SMS "4821"
POST /otp/verify  -> GET otp:98765
                       ├─ "4821" matches -> DEL otp:98765 -> login OK
                       └─ (nil)          -> "OTP expired"
```

15. Use 4: Rate limiting (stop abuse)

```
Rule: max 5 login attempts per minute per IP
request ──> INCR rate:login:1.2.3.4 ──> value = 1? then EXPIRE 60
                    |
          value <= 5 ──> allow
          value  > 5 ──> 429 Too Many Requests  (key auto-resets after 60 s)
```

```
const count = await this.redis.incr(`rate:login:${ip}`);
if (count === 1) await this.redis.expire(`rate:login:${ip}`, 60);
if (count > 5) throw new HttpException('Too many attempts', 429);
```

16. Use 5: Job queue (BullMQ, #57)

```
API ──add job──> Redis list "bull:emails:wait" [job1][job2][job3] ──> Worker pods take jobs
                                                        |
                                        KEDA watches the length and scales the workers
```

17. Use 6: Leaderboard

```
ZADD leaderboard 980 Ravi       sorted set keeps itself ordered:
ZINCRBY leaderboard 50 Asha        1. Ravi   980
ZREVRANGE leaderboard 0 9          2. Asha   920   <- moved up
                                   3. Kiran  760
Top 10 among 1 million players in about 1 ms (no ORDER BY on the DB)
```

18. Use 7: Pub/Sub (live messages)

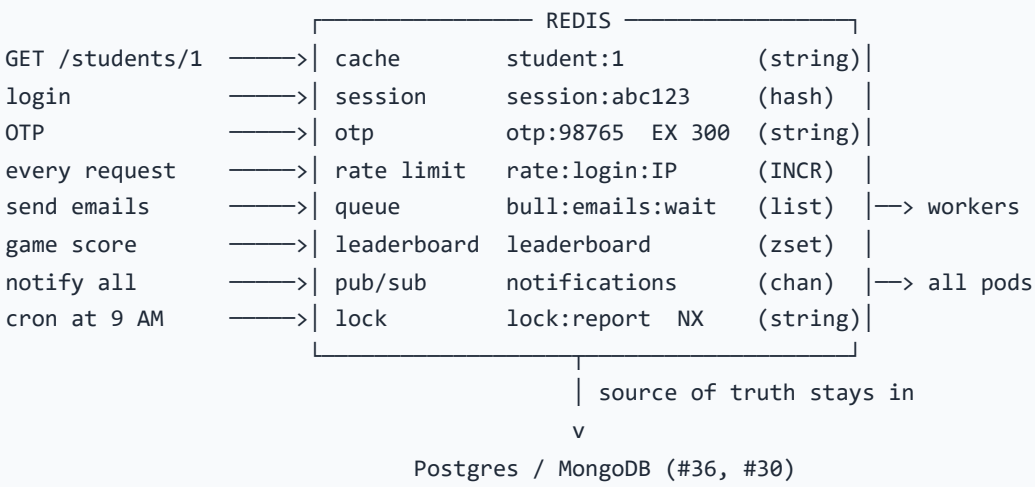
```
Publisher                    Redis channel "notifications"          Subscribers
_________                    _____________________________          ___________

PUBLISH notifications  ─────>          •  ──────────────────────>  API pod 1 -> WebSocket users
"Fees due tomorrow"                    •  ──────────────────────>  API pod 2 -> WebSocket users
                                       •  ──────────────────────>  API pod 3 -> WebSocket users
Fire-and-forget: if nobody listens, the message is lost (use Streams to keep history)
```

19. Use 8: Distributed lock (one pod only)

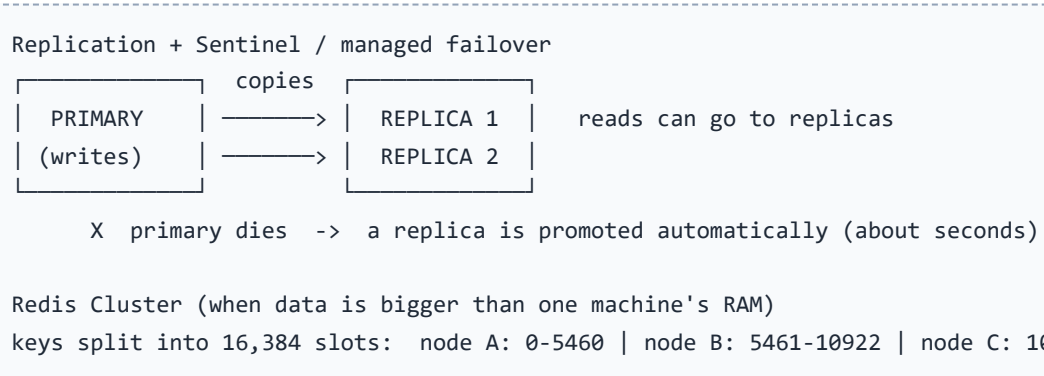
```
9:00 AM cron in 3 pods (#58):
  pod 1: SET lock:report pod1 NX EX 300  -> OK   -> runs the report
  pod 2: SET lock:report pod2 NX EX 300  -> nil  -> skips
  pod 3: SET lock:report pod3 NX EX 300  -> nil  -> skips
NX = only set if it doesn't exist. EX = auto-unlock if pod 1 crashes.
```

20. All uses in one picture



PART 5: Production picture

21. High availability



22. Redis vs a normal database

	Redis	Postgres / Mongo
Where data lives	RAM (+ optional disk copy)	Disk (+ memory cache)
Speed	About 0.1-1 ms	About 1-50 ms
Size	Limited by RAM (GBs)	TBs
Queries	By key only (no JOIN, no WHERE)	Rich SQL / filters
Main role	**Speed layer**: cache, temp data, counters	**Source of truth**

23. Common mistakes

Mistake	Fix
Using Redis as the only copy of important data	Keep the truth in Postgres/Mongo. Redis is the fast copy.
No TTL on cache keys	RAM fills up with stale data. Always set `EX`.
`KEYS *` in production	Blocks the single thread. Use `SCAN`.
Huge values (50 MB JSON)	Slow for everyone. Keep values small, and big files go in S3.
Forgetting to delete the cache on update	Users see old data. `DEL` the key after a DB update.
Redis open to the internet without a password	Bind to a private network, and set a password/ACL and TLS.
Trying to install Redis natively on Windows	Use WSL 2 or Docker.

24. Summary

- **Redis = REmote DIctionary Server**: key → value data in **RAM**, answers in under 1 ms.
- Types: **string, hash, list, set, sorted set, stream, pub/sub**.
- Inside: **one thread + event loop**. Fast, atomic commands, but avoid slow commands (`KEYS *`).
- Persistence: **RDB** snapshots and/or an **AOF** log. Full RAM: an **eviction** policy (e.g. allkeys-lru).
- OS: **Linux**. On Windows, use **WSL 2** or **Docker**. In production, **managed** (ElastiCache).
- Uses: **cache, sessions, OTP, rate limit, queue, leaderboard, pub/sub, locks**.
- Rule: **Postgres/Mongo = truth, Redis = speed**.